

Maria Luiza Kodama RA: 823128136 n

Caio Coelho Jatoba RA: 823160685

Atividade Proposta para a próxima última aula

• Pesquisar, Implementar, Codificar e Verificar Algoritmos:

❑ Linguagem de Programação:

- Java ou Python;

❑ Algoritmos:

- Criptografias com Chaves Simétricas;

- Criptografias com Chaves Assimétricas;

- Função Hash.

Python

1. Criptografia com Chave Simétrica (AES)

Nesse tipo de criptografia, a mesma chave é usada para codificar e decodificar a mensagem. É rápido e simples, mas exige que você compartilhe a chave com segurança.

A gente vai usar o módulo `cryptography`, que é bem completo e fácil de usar.

```
from cryptography.fernet import Fernet

# Gerando uma chave secreta
chave = Fernet.generate_key()
cipher = Fernet(chave)

# Mensagem que queremos proteger
mensagem = b"Essa é uma mensagem secreta."

# Criptografando
mensagem_criptografada = cipher.encrypt(mensagem)

# Descriptografando
mensagem_original = cipher.decrypt(mensagem_criptografada)

print(" Chave:", chave)
print(" Criptografada:", mensagem_criptografada)
print(" Descriptografada:", mensagem_original.decode())
```

2. Criptografia com Chave Assimétrica (RSA)

Aqui usamos **duas chaves**: uma pública (pra criptografar) e outra privada (pra descriptografar). É o que usamos em coisas como certificados digitais, login seguro, etc.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Criando o par de chaves
chave_privada = rsa.generate_private_key(public_exponent=65537, key_size=2048)
chave_publica = chave_privada.public_key()

mensagem = b"Mensagem protegida com RSA"

# Criptografando com a chave pública
mensagem_criptografada = chave_publica.encrypt(
    mensagem,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Descriptografando com a chave privada
mensagem_original = chave_privada.decrypt(
    mensagem_criptografada,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print(" Criptografada:", mensagem_criptografada)
print(" Descriptografada:", mensagem_original.decode())
```

3. Funções Hash (SHA-256)

Diferente da criptografia, uma função hash **não pode ser revertida**. Ela serve para verificar integridade, por exemplo, pra saber se um arquivo ou senha foi alterado.

```
import hashlib
mensagem = "Confiança e integridade com SHA-256"
hash_gerado = hashlib.sha256(mensagem.encode()).hexdigest()
print("✉ Mensagem:", mensagem)
print("📄 Hash SHA-256:", hash_gerado)
```